

# BINARY SEARCH TREES — PART 1

*"What if a tree could find anything in  
seconds, just by being organized?"*



## ■ TODAY'S CLASS ■

Last class we walked trees in four different orders. Today we put an **ORDER RULE** inside the tree itself. That single rule unlocks search in  $O(\log n)$  — and gives us fast insert too.

| Block                   | What we cover                      |
|-------------------------|------------------------------------|
| <b>Hook</b>             | Why a sorted array is not enough   |
| <b>The BST property</b> | One rule, everywhere in the tree   |
| <b>Search in action</b> | Trace a target through the tree    |
| <b>Complexity</b>       | $O(h)$ — and what `h` really means |
| <b>Recursive search</b> | Code + base/inductive cases        |
| <b>Recursive insert</b> | Code + tracing six values          |
| <b>Big picture</b>      | BST vs sorted array vs degenerate  |

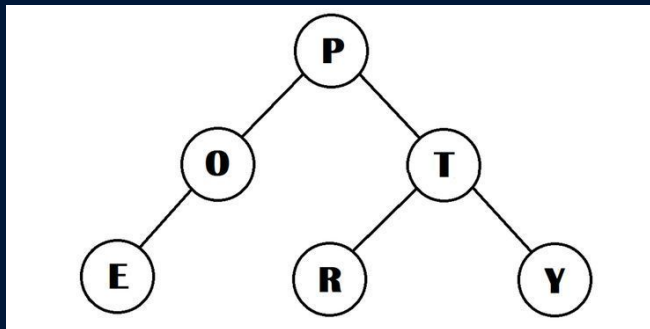
*"One rule, applied everywhere — and we get both fast search and fast insert."*

## ■ WHY WE NEED BST AS A DATA STRUCTURE ■

Your WeChat contact list has 5 000 people. With an unsorted list you scroll forever —  $O(n)$ .

With a sorted list you binary search —  $O(\log n)$ . But adding a new contact in a sorted array means shifting everything —  $O(n)$ .

We want a structure that gives us **fast search AND fast insert** — the **Binary Search Tree** does both, by combining binary-search speed with tree flexibility.



### [01] Unsorted list

Search:  $O(n)$ . Insert:  $O(1)$ .

*Search is unbearable.*

### [02] Sorted array

Search:  $O(\log n)$ . Insert:  $O(n)$ .

*Insert is unbearable.*

### [03] Binary Search Tree

Search:  $O(\log n)$ . Insert:  $O(\log n)$ .

*Both fast — if the tree stays balanced.*

*"Pick a structure by the operations you do most. BST is the answer when you do both — a lot."*

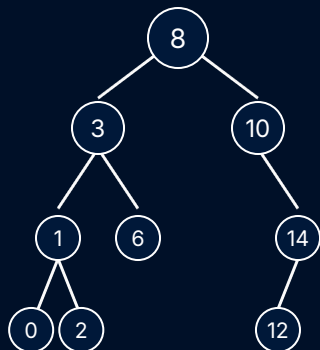
# ■ ONE RULE, EVERYWHERE ■

A Binary Search Tree is a binary tree where every node follows one rule.

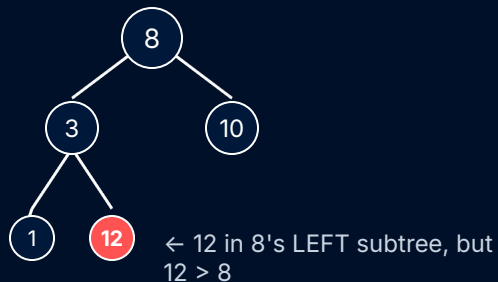
**All values in the LEFT subtree are smaller** than the node.

**All values in the RIGHT subtree are greater.** That's it — applied everywhere, recursively.

✓ Valid BST



✗ NOT a BST



The BST property is recursive. Every subtree of a valid BST is itself a valid BST.

*"Left < node < right. Same rule at every level. One rule unlocks everything."*

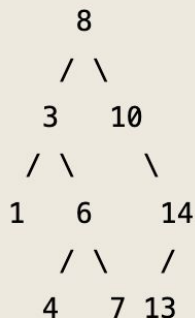
## ■ ELIMINATE HALF EVERY STEP ■

To search a value, we never look everywhere. At each node we compare.

Smaller → go left. Greater → go right. Equal → found.

**Each comparison throws away half the tree.**

Search for 7 in this BST:



Step 1: at 8,  $7 < 8 \rightarrow$  LEFT

Step 2: at 3,  $7 > 3 \rightarrow$  RIGHT

Step 3: at 6,  $7 > 6 \rightarrow$  RIGHT

Step 4: at 7 → FOUND ✓

4 comparisons in a tree of 8 nodes.

### Sounds familiar?

It is **binary search**, but walking a tree instead of indexing into an array.

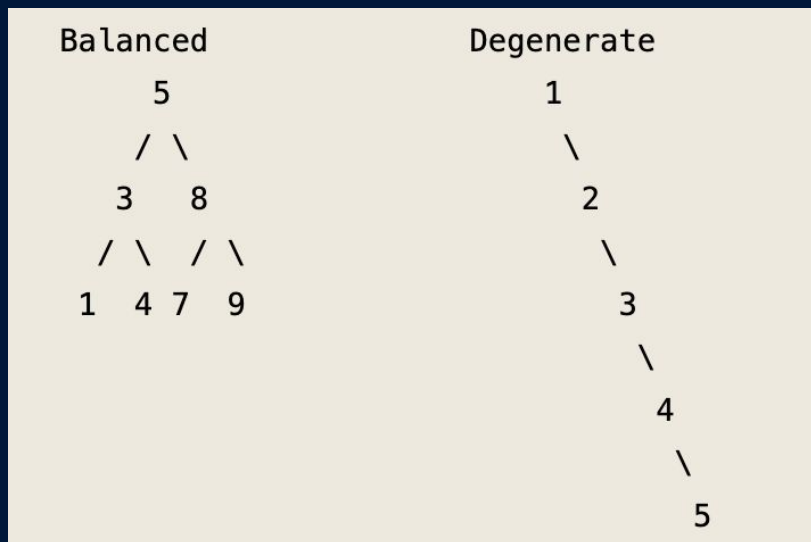
The order property is what makes the half-elimination work.

*"At every node, one comparison kills half the tree. That is the power of order."*

## ■ SHAPE DECIDES SPEED ■

Search cost is  $O(h)$ , where  $h$  is the height of the tree. So the question is: how tall is the tree?

A balanced tree is  $O(\log n)$  tall. A degenerate tree is  $n$  tall. Same values, different shapes, very different speeds.



### [01] Balanced BST

Height  $\sim \log_2(n)$ .

Search cost  $O(\log n)$  ✨

### [02] Degenerate (chain)

Height  $n$ . Search cost  $O(n)$  😞

*" $O(h)$  –  $h$  is the height. Keep the tree short and the BST flies. Let it stretch and it crawls."*

## ■ SEARCH, IN PYTHON ■

Three cases. Two base cases — empty subtree, or value matches. One inductive case — compare, then recurse on the correct side. Exactly the pattern from the last class on trees, with a comparison driving the direction.

```
def search(node, target):  
    if node is None: # base case 1: empty  
        return None  
    if node.value == target: # base case 2: match  
        return node  
    if target < node.value: # recurse left  
        return search(node.left, target)  
    else: # recurse right  
        return search(node.right, target)
```

### No loop.

The recursion does the walking. The comparison decides the direction.

One return path per case.

*"Two base cases, one comparison, one recursion. Search is shorter than the description."*

## ■ INSERT, IN PYTHON ■

To insert a value, we search for it. When we hit an empty spot — **None** — that's where the new node goes. The tricky part is the return value: we **re-attach the returned subtree** on the way back up.

```
def insert(node, value):  
    if node is None:  
        # base case: empty spot  
        return Node(value)  
  
    if value < node.value:  
        node.left = insert(node.left, value)  
    elif value > node.value:  
        node.right = insert(node.right, value)  
  
    # if value == node.value, do nothing  
    return node # ⚠ always return the node
```

### [01] Why return `node`?

The caller re-assigns the link. We must return the subtree root for re-attachment.

### [02] The base case creates

When we hit `None`, we make a new `Node` and hand it back up the call stack.

### [03] Duplicates ignored

If values match, we fall through and return the node unchanged.

*"Walk down comparing, plant the new node when you hit None, re-attach on the way back up."*



## ■ FROM EMPTY TO TREE ■

Insert the values **[5, 3, 8, 1, 4, 7]** into an empty BST, in order. Each insertion follows the same algorithm. Notice how the shape grows.

insert 5

5

insert 3

5

/

3

insert 8

5

/ \

3

8

insert 1

5

/ \

3

8

/

1

insert 4

5

/ \

3

8

/ \

1

4

insert 7

5

/ \

3

8

/ \ /

1

4

7

### [01] Order Matters

The same values in a different order would give a different shape.

Insert **[1, 2, 3, 4, 5, 6, 7]** and you get a chain — a **degenerate BST**. We come back to this next class.

*"Same content, different order of arrival, different tree. Shape depends on history."*

## ■ BST VS THE WORLD ■

Compare the three options for "store sorted values and search them later". A balanced BST wins on every operation we care about. A degenerate BST loses every operation. Shape is everything.

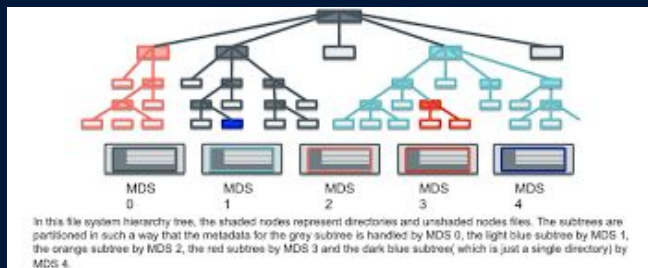
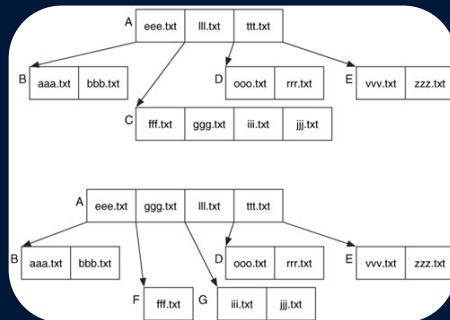
| Operation | BST (balanced) | BST (degenerate) | Sorted array |
|-----------|----------------|------------------|--------------|
| Search    | $O(\log n)$ ✨  | $O(n)$           | $O(\log n)$  |
| Insert    | $O(\log n)$ ✨  | $O(n)$           | $O(n)$       |
| Delete    | $O(\log n)$ ✨  | $O(n)$           | $O(n)$       |

A BST gives you **fast search AND fast insert** — as long as it stays balanced. Keeping it balanced is the topic of next class.

*"Balanced BST = binary search made dynamic. You can keep adding and removing without losing speed."*

# ■ REAL-WORLD USES ■

BSTs are not academic curiosities. They (and their close cousins like B-trees, AVL, Red-Black) power systems you use every day.



## [01] Database indexes

B-trees (a generalisation of BST) are how databases find a row by key in milliseconds, across millions.

## [02] Auto-complete and dictionary lookup

Fast prefix and exact-match search on a sorted set of words.

## [03] Range queries

"All users between age 18 and 25" — BSTs naturally walk an ordered range.

## [04] File systems metadata

Inode trees, directory structures — many file systems are tree-based for fast lookup.

*"Every fast key-value lookup you have ever made is, somewhere underneath, a tree like this one."*

# ■ WRAP-UP ■

## What you now own

- **The BST property:** left < node < right, everywhere.
- **Recursive search:** two base cases + one comparison + one recursion.
- **Recursive insert:** same walk, plant a node at the empty spot, re-attach on the way up.
- Complexity is  **$O(h)$**  — balanced means  $O(\log n)$ , degenerate means  $O(n)$ .
- **BST = binary search made dynamic.**

## Next class (class 08)

- **Delete** in a BST — three cases, the last one is the tricky one.
- The **degenerate** BST problem: insert sorted values and the tree collapses into a chain.
- **Balanced trees** (AVL, Red-Black) — the concept, not the heavy implementation.
- Why "balanced" is not a luxury — it is the line between  **$O(\log n)$**  and  **$O(n)$** .

*"Today: one rule unlocks fast search. Next class: what to do when that rule lets the tree collapse."*